CPP (Fundamentals)

1) Basic I/O Setup

```
ios::sync_with_stdio(false);
cin.tie(nullptr);
```

↳ disable stream synchronizations

2) Basic Input types

float/ double	`double x;`	`cin >> x;`	`cout << fixed << setprecision(6) << x;` (fixed shows in non scientific format)
string (no spaces)	`string s;`	`cin >> s;`	`cout << x;`
line with spaces	`string line;`	`getline(cin, line);`	`cout << line;`

fixed array

```
int n;
cin >> n;
int arr[n];
for(int i=0; i<n, i++) cin >> arr[i];
```

2D Input/ Matrix

```
int n, m;
cin >> n >> m;
vector<vector<int>> mat(n, vector<int> m);
    for(int i=0; i<n; i++){
```

```
for(int j=0; j<n; j++){
    cin >> mat[i][j];
```

3) EOF

```
int x; ↳ Used when number of testcase are unknown
while ( cin >> x){
    // process x
}
```

4) Split in CPP

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    string line;
getline( cin, line); ↳ Input like 1 2 3 4 5 6
istringstream iss(line); ↳ creates a fake input stream
vector<int> nums;
int x;
while ( iss >> x){
    nums.push_back (x);
}
}
```

→

```cpp
#include < bits/stdc++.h>
using namespace std;

int main(){

    string line;
    getline(cin, line); ~> 1,2,3,4,5,6

    vector<int> nums;
    stringstream ss(line); ~>create fake stream but string can
                           do both input/output
    string token;
    while(get(ss, token, ',')){

        nums.push_back(stoi(token));

    }

}
```

5) Type Def

string → int	stoi(s)
string → long long	stoll(s)
string → float	stof(s)
string → double	stod(s)
int → string	to_string(x)

6) Map ↳ done using transform

```cpp
vector<int> nums = {1,2,3,4,5};
vector<int> doubled(nums.size());

transform(nums.begin(), nums.end(), doubled.begin(),
          [](int x){
    return x*2;
});
```

7) filter

↳ done using copy_if()

```
vector<int>nums = {1,2,3,4,5,6}
vector<int> evens;
copy_if(nums.begin(), nums.end(), back_inserter(evens),[](int x){
        return x%2==0;
});
```

Deque (Modifier)

1) clear

```
int main(){
    deque<int> d = {1,2,3};
    d.clear();
    cout << d.size(); //0
}
```

2) insert

```
deque<int> d = {1,3,4};
d.insert(d.begin()+1,2); //{1,2,3,4}
```

3) insert_range() [C++23]

```
deque<int> d = {1,2};
vector<int> v = {3,4,5};
d.insert_range(d.end(),v); //[1,2,3,4,5]
```

4) erase()

```
deque<int> d = {1,2,3,4};
d.erase(d.begin()+1); // remove 2 → [1,3,4]
d.erase(d.begin(), d.begin()+2); // remove first two → [4]
```

5) push_back()

```
deque<int> d={1,2};
d.push_back(3); // [1,2,3]
```

6) Append_range [C++23]

```
deque<int>d = {1};
vector<int> v = {2,3,4};
d.append_range(v); // [1,2,3,4];
```

7) Pop_back()

```
deque<int>d= {1,2,3};
d.pop_back() // [1,2]
```

8) Push_front

```
deque<int> d = {2,3};
d.push_front(1); //[1,2,3]
```

9) prepend_range

```
deque<int> d = {3,4}
vector<int> v = {1,2};
d.prepend_range(v); //[1,2,3,4]
```

10) push_front()

```
deque<int>d = {2,3};
d.push_front(1); //[1,2,3];
```

11) Pop_front

```
deque<int>d = {1,2,3};
d.pop_front(); //[2,3];
```

12) resize()

```
deque<int> d = {1,2,3};
d.resize(5,0) //[1,2,3,0,0]
```

13) swap()

```
deque<int> a = {1,2};
deque<int>b ={3,4}
a.swap(b); // a={3,4}, b=[1,2}
```

11) Set

↳ Store unique element (no duplicate)

↳ keep them sorted automatically

↳ uses a balanced BST (RB Tree) → O(logn) insertion

```
#include <iostream>
# include < set>
using namespace std;
int main(){
    set<int> s = {3,1,4};
    s.insert(2);
    for ( int x:s) cout<<x<<" "; // Output - 1 2 3 4
}
```

Iterating over a Set

1) Range-based for loop

```
for(int x:s) cout<<x<<" ";
```

2) Reverse Iterator

```
for(auto it = s.rbegin(); it!=s.rend(); it++){
   cout << *it<<" ";
}
```

1) count()

↳ returns 1 if the element exist, otherwise 0.

```
cout << s.count(20)<<endl; → return 1 (exists)
cout << s.count(25)<<endl; → return 0 (does not exist)
```

↳ check quickly if an element (O log n).

2) find()

↳ find an element and return an iterator to it (or s.end())
if not found.

```
auto it = s.find(30);
if (it != s.end()){
    cout << "found: " << *it << endl;
}else{
    cout << "Not found" << endl;
```

3) contain()

↳ simpler version of find()

```
if (s.contain(40)){
    cout << " Set contains 40" << endl;
}else{
    cout << " No 40" << endl;
```

4) equal_range()

↳ return a pair of iterators - lower bound and upper bound for elements equal to a given key.

```
auto [low, hi] = s.equal_range(2);
```

5) lower_bound(x)

6) upper_bound(x)

iv) Map

↳ Map is a sorted associative container that contains key-value pair with unqui keys.

1) clear

↳ Remove all elements from the map, leaving it with a size of 0.

```
scores.clear();
```

2) insert

↳ insert a new key-value pair. if the key already exist insert does nothing. it returns a std::pair containing an iterator to the element (either the newly inserted one or the existing one) and a bool which is true if the insertion took place, & false otherwise.

```
int main() {
    std::map<std::string, int> scores = {{"Alice", 95}};
    print_map(scores);

    // Insert a new element - succeeds
    auto result1 = scores.insert({"Bob", 88});
    std::cout << "Inserting 'Bob': success = " << std::boolalpha << result1.second << std::endl;
    print_map(scores);

    // Try to insert an existing element - fails
    auto result2 = scores.insert({"Alice", 100});
    std::cout << "Inserting 'Alice' again: success = " << result2.second << std::endl;
    print_map(scores); // Alice's score remains 95
    return 0;
}
```

```
{ 'Alice': 95 }
Inserting 'Bob': success = true
{ 'Alice': 95 'Bob': 88 }
Inserting 'Alice' again: success = false
{ 'Alice': 95 'Bob': 88 }
```

3) insert_range

↳ insert a range of element from another container more efficient than inserting element one by one in a loop.

```
int main() {
    std::map<std::string, int> scores = {{"Alice", 95}};
    std::vector<std::pair<std::string, int>> new_scores = {{"Charlie", 92}, {"David", 78}};

    std::cout << "Before insert_range: ";
    print_map(scores);

    scores.insert_range(new_scores);

    std::cout << "After insert_range: ";
    print_map(scores);
    return 0;
}
```

```
Before insert_range: { 'Alice': 95 }
After insert_range: { 'Alice': 95 'Charlie': 92 'David': 78 }
```

insert_or_assign (C++17)

↳ This is a very useful function. it inserts a key-value pair if the key does not exist, it updates the value associated with that key. This avoids the common "find-then-update or insert" pattern

```
int main() {
    std::map<std::string, int> scores = {{"Alice", 95}};
    print_map(scores);

    // Insert a new element
    scores.insert_or_assign("Bob", 88);
    std::cout << "After inserting 'Bob': ";
    print_map(scores);

    // Update an existing element
    scores.insert_or_assign("Alice", 100);
    std::cout << "After assigning 'Alice': ";
    print_map(scores);
    return 0;
}
```

```
{ 'Alice': 95 }
After inserting 'Bob': { 'Alice': 95 'Bob': 88 }
After assigning 'Alice': { 'Alice': 100 'Bob': 88 }
```

erase

↳ Remove element from the map. it can be used in three ways:

1) By key → Removes the element with the specified key.
2) By iterator → Removes the element at the iterator's problem
3) By range all element in the range [first, last).

```cpp
int main() {
    std::map<std::string, int> scores = {{"Alice", 95}, {"Bob", 88}, {"Charlie", 92}, {"David", 78}};
    print_map(scores);

    // 1. Erase by key
    scores.erase("Bob");
    std::cout << "After erasing 'Bob': ";
    print_map(scores);

    // 2. Erase by iterator
    auto it = scores.find("David");
    if (it != scores.end()) {
        scores.erase(it);
    }
    std::cout << "After erasing 'David': ";
    print_map(scores);
    return 0;
}
```

```
{ 'Alice': 95 'Bob': 88 'Charlie': 92 'David': 78 }
After erasing 'Bob': { 'Alice': 95 'Charlie': 92 'David': 78 }
After erasing 'David': { 'Alice': 95 'Charlie': 92 }
```

Swap

↳ Exchange the content of two maps. This is an extreamely fast operation (O(1)) because it just swap internal pointer, not the actual element.

```cpp
int main() {
    std::map<std::string, int> map1 = {{"A", 1}, {"B", 2}};
    std::map<std::string, int> map2 = {{"X", 10}, {"Y", 20}};

    std::cout << "Map1 before swap: "; print_map(map1);
    std::cout << "Map2 before swap: "; print_map(map2);

    map1.swap(map2);

    std::cout << "Map1 after swap: "; print_map(map1);
    std::cout << "Map2 after swap: "; print_map(map2);
    return 0;
}
```

```
Map1 before swap: { 'A': 1 'B': 2 }
Map2 before swap: { 'X': 10 'Y': 20 }
Map1 after swap: { 'X': 10 'Y': 20 }
Map2 after swap: { 'A': 1 'B': 2 }
```

extract (C++17)

↳ Removes a node (key value) Pair from the map without destroying it. It returns a 'node handle' that own the element. This can be used to re-insert into the same map or another

compatible map, which is efficient.

```
int main() {
    std::map<std::string, int> scores = {{"Alice", 95}, {"Bob", 88}};

    // Extract the node for "Bob"
    auto node_handle = scores.extract("Bob");

    std::cout << "Scores after extracting 'Bob': ";
    print_map(scores);

    // We can modify the value in the extracted node
    node_handle.mapped() = 90;

    // Insert the modified node back into the map
    scores.insert(std::move(node_handle));

    std::cout << "Scores after inserting node back: ";
    print_map(scores);
    return 0;
}
```

move is required as node_handle is movable but can't be copied.

```
Scores after extracting 'Bob': { 'Alice': 95 }
Scores after inserting node back: { 'Alice': 95 'Bob': 90 }
```